

Lekcja

Temat: Transakcja w MySQL

Transakcja w MySQL to grupa operacji SQL wykonywana jako jedna całość.

Transakcja spełnia właściwości **ACID**:

- **A — Atomicity (atomowość)**
Wszystko albo nic. Jeśli jedna operacja się nie uda → cofamy wszystko.
- **C — Consistency (spójność)**
Dane po transakcji muszą być poprawne.
- **I — Isolation (izolacja)**
Transakcje nie przeszkadzają sobie nawzajem.
- **D — Durability (trwałość)**
Po COMMIT dane zostają zapisane na stałe.

Instrukcja

START TRANSACTION

BEGIN

COMMIT

ROLLBACK

SAVEPOINT

ROLLBACK TO SAVEPOINT

RELEASE SAVEPOINT

SET TRANSACTION

Opis

Rozpoczyna transakcję

Skrót do START TRANSACTION

Zatwierdza zmiany

Wycofuje zmiany

Tworzy punkt przywracania

Cofnięcie do punktu

Usunięcie savepoint

Ustawienie parametrów transakcji

Przykład 1:

```
CREATE TABLE konto_bankowe (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  wlasciciel VARCHAR(100),  
  saldo DECIMAL(10,2)  
);  
INSERT INTO konto_bankowe (wlasciciel, saldo)  
VALUES  
  ('Jan', 5000),  
  ('Anna', 3000);
```

```
/*
Od tego momentu zmiany nie są jeszcze zapisane na stałe.
*/
START TRANSACTION;
select * from konto_bankowe;

UPDATE konto_bankowe
SET saldo = saldo - 1000
WHERE wlasciciel = 'Jan';

UPDATE konto_bankowe
SET saldo = saldo + 1000
WHERE wlasciciel = 'Anna';
select * from konto_bankowe;

COMMIT; -- Zmiany zostają zapisane na stałe.
```

Przykład 2 błędu i ROLLBACK:

```
START TRANSACTION;

select * from konto_bankowe;

UPDATE konto_bankowe
SET saldo = saldo - 10000
WHERE wlasciciel = 'Jan';
-- Jan ma teraz błędne saldo.

ROLLBACK; -- cofa wszystko od START TRANSACTION
select * from konto_bankowe;
```

ROLLBACK bez SAVEPOINT:

ROLLBACK:

- cofa WSZYSTKIE zmiany,
- od ostatniego:
 - START TRANSACTION
 - lub BEGIN.

Przykład 3 cofnięcia za pomocą SAVEPOINT:

```
START TRANSACTION;
```

```
UPDATE konto SET saldo = saldo - 100 WHERE id = 1;
SAVEPOINT s1;

UPDATE konto SET saldo = saldo + 100 WHERE id = 2;
ROLLBACK TO SAVEPOINT s1;

COMMIT;
```

Przykład 4 po COMMIT rollback już nie działa

```
START TRANSACTION;

UPDATE konto
SET saldo = 0;
COMMIT;

ROLLBACK; -- NIE zadziała
```

Autocommit

```
SET autocommit = 1; -- każda instrukcja jest automatycznie commitowana
```

W MySQL domyślnie włączony jest tryb **autocommit=1**.

Oznacza to, że każda pojedyncza instrukcja **INSERT/UPDATE/DELETE** jest traktowana jako osobna transakcja. **START TRANSACTION** lub **BEGIN** tymczasowo wyłącza autocommit aż do **COMMIT/ROLLBACK**.

Lekcja [Tej lekcji jeszcze nie było, dopracować]

Temat: Bezpieczeństwo w MySQL – kluczowe obszary

Transparent Data Encryption (TDE)

Transparent Data Encryption (TDE) to mechanizm szyfrowania danych „w spoczynku” (data at rest), który zapewnia ochronę danych zapisanych na dysku. Dane są automatycznie szyfrowane

podczas zapisu i odszyfrowywane podczas odczytu, bez konieczności modyfikacji zapytań SQL przez użytkownika.

Z perspektywy użytkownika i aplikacji proces szyfrowania jest całkowicie transparentny.

Zakres szyfrowania TDE:

- pliki tabel InnoDB,
- logi redo/undo,
- pliki tymczasowe,
- backupy (w zależności od konfiguracji).

Ochrona danych przed nieautoryzowanym dostępem na poziomie systemu plików (np. kradzież dysku, kopii zapasowej lub dostęp administracyjny do plików bazy).

Ograniczenia środowiska XAMPP (MariaDB 10.4)

W środowisku MariaDB działającym w XAMPP:

```
SELECT VERSION();  
-- 10.4.32-MariaDB
```

występują ograniczenia w zakresie monitorowania i zarządzania szyfrowaniem na poziomie tablespace.

Próba odczytu metadanych:

```
SELECT *  
FROM information_schema.INNODB_TABLESPACES;
```

zwraca błąd:

```
#1109 - Unknown table 'innodb_tablespaces' in information_schema
```

Oznacza to brak dostępności tej struktury systemowej w danej wersji środowiska, co uniemożliwia pełne monitorowanie szyfrowania na poziomie tablespace.

Key Management i ograniczenia TDE w XAMPP

W środowisku XAMPP dostępna jest składnia:

```
ENCRYPTION = 'Y'
```

jednak jej pełne wykorzystanie wymaga systemu zarządzania kluczami (key management), który nie jest dostępny w domyślnej instalacji MariaDB XAMPP.

Weryfikacja dostępnych pluginów:

```
SHOW PLUGINS;
```

Brak następujących komponentów oznacza brak systemu zarządzania kluczami:

- `file_key_management`

- keyring_file
- innodb_key_management

W konsekwencji mechanizm TDE nie może działać w pełnym zakresie, mimo dostępności składni SQL.

Zastosowane podejście – szyfrowanie kolumn (AES)

W związku z ograniczeniami środowiska zastosowano szyfrowanie na poziomie aplikacyjnym przy użyciu funkcji:

- AES_ENCRYPT()
- AES_DECRYPT()

Przykład 1

```
CREATE TABLE users (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    pesel BLOB );
```

```
INSERT INTO users (pesel) VALUES  
(AES_ENCRYPT('12345678901', 'secret_key'));
```

```
SELECT AES_DECRYPT(pesel, 'secret_key')  
FROM users;
```

```
SELECT CAST(AES_DECRYPT(pesel, 'secret_key') AS CHAR) FROM users;
```

Przykład 2

```
CREATE TABLE customers (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100),  
    pesel VARBINARY(255),  
    credit_card VARBINARY(255));
```

```
INSERT INTO customers (name, pesel, credit_card) VALUES  
( 'Jan Kowalski', AES_ENCRYPT('12345678901', 'secret_key'), AES_ENCRYPT('1111-2222-  
3333-4444', 'secret_key') );
```

```
SELECT name,  
    CAST(AES_DECRYPT(pesel, 'secret_key') AS CHAR) AS pesel,  
    CAST(AES_DECRYPT(credit_card, 'secret_key') AS CHAR) AS credit_card  
FROM customers;
```

W środowisku XAMPP opartym o MariaDB 10.4 mechanizmy TDE są ograniczone ze względu na brak systemu zarządzania kluczami oraz brak pełnej obsługi metadanych `tablespace`. W związku z tym w analizie bezpieczeństwa zastosowano szyfrowanie na poziomie kolumn przy użyciu funkcji `AES_ENCRYPT()` i `AES_DECRYPT()`, które zapewniają ochronę danych w spoczynku na poziomie logicznym.

Lekcja

Temat: SQL Injection

SQL Injection (SQLi) to rodzaj ataku na aplikacje webowe, w którym ktoś wstrzykuje własny kod SQL do zapytania bazy danych, żeby zmienić jego działanie.

SQL Injection (SQLi) to rodzaj ataku na aplikacje webowe, w którym ktoś **wstrzykuje własny kod SQL do zapytania bazy danych**, żeby zmienić jego działanie.

Przykład zastosowania:

```
CREATE TABLE uzytkownik (  
id INT PRIMARY KEY AUTO_INCREMENT,  
login VARCHAR(50),  
haslo VARCHAR(50)  
);
```

```
INSERT INTO uzytkownik (login, haslo) VALUES ('admin', '1234');
```

```
INSERT INTO uzytkownik (login, haslo) VALUES ('jan', 'abcd');
```

```
INSERT INTO uzytkownik (login, haslo) VALUES ('anna', 'pass123');
```

Poprawne dane logowania. Aplikacja buduje zapytanie:

```
SELECT * FROM users WHERE login = 'admin' AND password = '1234';
```

SQL Injection (atak)

Atakujący wpisuje:

- login: admin' --
- hasło: cokolwiek (np. xyz)

Aplikacja wykonuje

Wstawia to do zapytania:

```
SELECT * FROM users WHERE login = 'admin' -- ' AND password = 'xyz';
```

Lekcja

Temat: SQL Injection – zabezpieczenie kodu w PHP

1. Prepared Statements (ochrona przed SQL Injection)

```
$stmt = $mysqli->prepare("SELECT username, password FROM users WHERE username = ?");  
$stmt->bind_param("s", $username);  
$stmt->execute();
```

To najważniejsze zabezpieczenie przeciwko atakom SQL Injection.

Zamiast wklejać dane użytkownika bezpośrednio do zapytania SQL (co pozwalało na klasyczne ' OR '1'='1'), kod używa **prepared statement** (parametryzowanego zapytania).

- ? to placeholder.
- bind_param("s", \$username) mówi bazie: „traktuj \$username jako zwykły string, nie jako kod SQL”.
- Serwer bazy danych rozdziela dane od kodu SQL, więc nawet jeśli ktoś wpisze ' OR '1'='1 – zostanie to potraktowane jako nazwa użytkownika, a nie warunek logiczny.

2. password_hash / password_verify (bezpieczne przechowywanie i weryfikacja haseł)

```
if ($user && password_verify($password, $user['password']))
```

- Hasła **nigdy nie są przechowywane w postaci jawnej**.
- W bazie znajdują się hashe wygenerowane przez password_hash(\$pass, PASSWORD_BCRYPT) (lub nowsze algorytmy).
- password_verify() porównuje podane hasło z hashem w bezpieczny sposób (timing-safe comparison).

Dlaczego to ważne?

- Nawet jeśli atakujący wykradnie całą bazę danych, nie dostanie haseł w formie czytelnej.
- Bcrypt automatycznie dodaje salt i jest wolny (co utrudnia brute-force na GPU).

3. Walidacja loginu (regex)

```
if (!preg_match('/^[a-zA-Z0-9_]{3,20}$/', $username))
```

Bardzo restrykcyjna biała lista (whitelist) dopuszczalnych znaków:

- Tylko litery (a-z, A-Z), cyfry (0-9) i podkreślenie _
- Długość od 3 do 20 znaków

Zalety:

- Uniemożliwia wprowadzenie niebezpiecznych znaków (np. ' , " , < , > , ; , -- itp.).
- Zmniejsza powierzchnię ataku (mniej wektorów do SQLi, XSS, Path Traversal itp.).
- Chroni przed wieloma rodzajami ataków na poziomie aplikacji.

4. CSRF Token (ochrona przed Cross-Site Request Forgery)

```
if (empty($_SESSION['csrf_token'])) {  
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));  
}  
  
// Sprawdzenie  
if (!isset($_POST['csrf_token']) || $_POST['csrf_token'] !== $_SESSION['csrf_token']) {  
    die("Nieprawidłowy token CSRF.");  
}
```

Atak CSRF polega na tym, że zalogowany użytkownik niechcący (np. przez złośliwy link lub obrazek) wykonuje akcję na stronie (w tym przypadku logowanie lub inną akcję POST)

Rozwiązanie:

- Generowany jest losowy, kryptograficznie silny token (32 bajty → 64 znaki hex).
- Token jest wysyłany jako ukryte pole w formularzu.
- Serwer sprawdza, czy token z formularza zgadza się z tym zapisanym w sesji.

Dodatkowo token jest generowany tylko raz na sesję (dobra praktyka).

5. Brute Force Protection (ochrona przed atakami siłowymi)

```
if (!isset($_SESSION['login_attempts'])) $_SESSION['login_attempts'] = 0;  
  
$_SESSION['login_attempts']++;  
  
if ($_SESSION['login_attempts'] >= 5) {  
    $_SESSION['blocked_until'] = time() + 300; // 5 minut  
}
```

- Licznik nieudanych prób logowania przechowywany w sesji.

- Po 5 nieudanych próbach konto (lub raczej sesja/IP) jest blokowane na 5 minut.
- Po poprawnym zalogowaniu licznik jest zerowany.

Zalety:

- Drastycznie spowalnia atak brute-force.
- Blokada czasowa (rate limiting) jest jedną z najskuteczniejszych metod.

Uwaga: W produkcji lepszym rozwiązaniem jest blokada na poziomie IP + account lockout + CAPTCHA po kilku próbach.

6. Bezpieczne komunikaty błędów

```
if ($mysqli->connect_error) {
    die("Błąd połączenia z bazą danych."); // nie pokazuje szczegółów
}
```

Zamiast pokazywać:

Warning: mysqli::prepare(): (HY000/1146) Table 'CTF.users' doesn't exist
 ...kod zwraca generyczny komunikat.

Szczegółowe błędy to skarbnica informacji dla atakującego (nazwa bazy, wersja MySQL, struktura tabel, ścieżki itp.).

7. htmlspecialchars (ochrona przed XSS)

```
$message = "... Witaj: <strong>" . htmlspecialchars($user['username']) . "</strong>";
htmlspecialchars() zamienia znaki specjalne:
```

- < → <
- > → >
- & → &
- " → "
- ' → '

Dzięki temu nawet jeśli login zawiera <script>alert(1)</script>, zostanie wyświetlony jako zwykły tekst, a nie wykonany kod JavaScript.